

Agent Phantom Recovery

07-trd.md

Technical Requirements Document (TRD)

Part 3

Infrastructure, Scalability, Reliability, Observability, Data & API Requirements

16. Infrastructure Requirements

16.1 Infrastructure Philosophy

The infrastructure must support long-horizon autonomous execution rather than short-lived request-response interactions.

The platform should be designed for:

- multi-step workflows
- persistent execution
- tool orchestration
- memory persistence
- repository analysis
- concurrent task processing

The infrastructure must remain modular so individual components can evolve independently.

16.2 High-Level Infrastructure Architecture

Frontend Workspace

↓

API Gateway

↓

Orchestrator Service

↓

Planner Service

Reasoner Service

Verifier Service
Memory Service
Tool Service
Execution Service
Repository Service

↓

Storage Layer

Each service must be independently deployable.

16.3 Core Infrastructure Components

Frontend Layer

Responsibilities:

- workspace rendering
 - browser view
 - code view
 - terminal view
 - chat console
 - task monitoring
-

API Layer

Responsibilities:

- request routing
 - authentication
 - authorization
 - rate limiting
 - session management
-

Orchestration Layer

Responsibilities:

- task lifecycle management
 - agent coordination
 - workflow control
 - execution tracking
-

Storage Layer

Responsibilities:

- persistence
 - memory storage
 - task history
 - logs
 - metadata
-

Execution Layer

Responsibilities:

- sandboxed execution
 - command execution
 - tool invocation
 - process monitoring
-

17. Service Architecture Requirements

17.1 Orchestrator Service

The orchestrator is the central control system.

Responsibilities:

- task scheduling
- state transitions
- workflow coordination
- retry handling
- execution supervision

The orchestrator should never perform deep analysis directly.

17.2 Planner Service

Responsibilities:

- goal decomposition
- plan generation
- task sequencing
- dependency management

Inputs:

- user goals
- memory context
- repository metadata

Outputs:

- executable plans
-

17.3 Reasoner Service

Responsibilities:

- code understanding
- root cause analysis
- patch generation
- architecture reasoning

Inputs:

- evidence
- repository context
- planner instructions

Outputs:

- findings
 - fixes
 - recommendations
-

17.4 Verifier Service

Responsibilities:

- review findings
- review patches
- identify regressions
- validate conclusions

Outputs:

- approve
 - reject
 - request additional evidence
-

17.5 Memory Service

Responsibilities:

- memory retrieval
- memory ranking
- memory storage
- context assembly

The memory service must operate independently from any single model.

17.6 Tool Service

Responsibilities:

- tool registration
- tool invocation
- permission enforcement
- execution monitoring

Supported tools:

- terminal
 - browser
 - file system
 - search
 - code execution
 - Git providers
-

17.7 Repository Service

Responsibilities:

- indexing
 - AST generation
 - dependency mapping
 - call graph creation
 - repository metadata generation
-

18. Storage Requirements

18.1 Storage Philosophy

The system must separate:

- operational data
- memory data
- repository data
- execution data

This separation improves scalability and maintainability.

18.2 Storage Categories

User Storage

Stores:

- account data
 - preferences
 - sessions
-

Task Storage

Stores:

- goals
 - plans
 - execution history
 - outcomes
-

Memory Storage

Stores:

- working memory
 - project memory
 - experience memory
-

Repository Storage

Stores:

- indexes

- embeddings
 - repository metadata
 - dependency graphs
-

Log Storage

Stores:

- tool outputs
 - execution traces
 - error events
 - audit records
-

18.3 Data Retention

The platform should support configurable retention policies.

Examples:

- temporary execution logs
- long-term project memory
- archived tasks

Retention should be adjustable by administrators.

19. Scalability Requirements

19.1 Scalability Philosophy

The architecture should scale across:

- users
- repositories
- tasks
- tool executions
- memory volume

The system should avoid architectural bottlenecks.

19.2 Horizontal Scaling

The following services should support horizontal scaling:

- API service

- orchestrator
 - planner
 - reasoner
 - verifier
 - memory service
 - tool service
-

19.3 Repository Scaling

The platform must support:

Small repositories:

< 10,000 files

Medium repositories:

10,000 - 100,000 files

Large repositories:

100,000+ files

Repository size should not directly impact reasoning cost.

Repository analysis pipelines should narrow context before model invocation.

19.4 Task Scaling

The platform should support:

- short tasks
- medium tasks
- long-running investigations

Long-running tasks should not block unrelated tasks.

19.5 Memory Scaling

Memory growth should be controlled through:

- summarization
- ranking

- archival
- deduplication

Memory systems should remain performant as data grows.

20. Reliability Requirements

20.1 Reliability Philosophy

The platform must continue operating even when individual components fail.

The system should favor graceful degradation over complete failure.

20.2 Fault Tolerance

The platform should tolerate failures in:

- model providers
- tools
- repositories
- execution environments
- network requests

Whenever possible, fallback behavior should be available.

20.3 Retry Management

Retries should support:

- exponential backoff
- maximum retry limits
- failure classification

Retries should be observable and logged.

20.4 Recovery Requirements

Recovery must preserve:

- task state
- plan state
- memory context
- execution history

A task should resume rather than restart whenever possible.

20.5 Failure Isolation

A failure in one subsystem should not cause:

- global outage
- task corruption
- memory corruption

Subsystem failures should remain isolated.

21. Observability Requirements

21.1 Observability Philosophy

Every important action should be observable.

The platform should make execution understandable to:

- users
 - developers
 - operators
-

21.2 Logging Requirements

The system shall log:

- API requests
 - tool invocations
 - model interactions
 - execution state transitions
 - verification outcomes
-

21.3 Metrics Requirements

The platform shall collect:

System Metrics

- CPU
- memory
- storage
- network

Agent Metrics

- task duration
- plan length
- tool usage
- verification success rate

Model Metrics

- latency
 - token usage
 - success rate
 - failure rate
-

21.4 Distributed Tracing

Each task shall have a trace identifier.

Trace data should include:

```
Task
↓
Plan
↓
Tool Usage
↓
Reasoning
↓
Verification
↓
Delivery
```

This enables debugging and auditability.

21.5 Execution Timeline

The platform should maintain a timeline showing:

- actions taken
- evidence gathered
- state transitions
- verification events

This timeline should be visible in the workspace.

22. Data Requirements

22.1 Data Classification

Data should be categorized as:

User Data

- profile
- settings
- preferences

Operational Data

- tasks
- plans
- execution traces

Repository Data

- indexes
- metadata
- analysis artifacts

Memory Data

- memories
 - summaries
 - embeddings
-

22.2 Data Integrity

The system shall protect against:

- corruption
- duplication
- inconsistent state

Data integrity checks should be automated.

22.3 Data Relationships

The platform should maintain relationships between:

- users
- tasks
- repositories
- memories

- tool executions
- verification results

These relationships will be important for future analytics.

23. API Requirements

23.1 API Philosophy

All major capabilities should be accessible through APIs.

This enables:

- frontend integration
 - future automation
 - external integrations
 - service communication
-

23.2 API Categories

Authentication APIs

Responsibilities:

- login
 - logout
 - token refresh
 - session management
-

Task APIs

Responsibilities:

- create task
 - update task
 - cancel task
 - retrieve task state
-

Agent APIs

Responsibilities:

- start execution
- pause execution

- resume execution
 - retrieve execution status
-

Memory APIs

Responsibilities:

- retrieve memories
 - update memories
 - delete memories
 - rank memories
-

Repository APIs

Responsibilities:

- ingest repository
 - retrieve analysis
 - retrieve file metadata
-

Tool APIs

Responsibilities:

- invoke tools
 - inspect tool status
 - retrieve tool output
-

23.3 API Design Standards

APIs shall be:

- versioned
- documented
- authenticated
- observable

Recommended standard:

`/api/v1/`

23.4 API Security

API access shall support:

- authentication
- authorization
- rate limiting
- audit logging

All sensitive operations must be authenticated.

24. Event Architecture Requirements

24.1 Event-Driven Design

The platform should use events for:

- task updates
 - tool completion
 - memory updates
 - verification completion
 - repository analysis completion
-

24.2 Core Events

Examples:

```
TaskCreated  
TaskStarted  
TaskPaused  
TaskCompleted  
TaskFailed
```

```
ToolStarted  
ToolCompleted  
ToolFailed
```

```
VerificationStarted  
VerificationPassed  
VerificationFailed
```

24.3 Event Persistence

Critical events should be stored for:

- auditing
 - debugging
 - analytics
 - replay
-

25. Integration Requirements

25.1 LLM Provider Integration

The architecture shall support multiple providers.

The system should not be tightly coupled to a single model vendor.

25.2 Git Provider Integration

Supported integrations should include:

- Git repositories
 - branch management
 - pull request workflows
-

25.3 Tool Integration Framework

New tools should be addable without major architectural changes.

Tool integration should be plugin-based.

25.4 Future Integration Support

Architecture should support future integrations such as:

- CI/CD systems
- issue trackers
- monitoring systems
- deployment platforms

without requiring major redesign.

End of TRD Part 3

Next Section:

- Performance Targets
- Deployment Requirements
- Security Operations Requirements
- Cost Management
- Technical Acceptance Criteria
- Production Readiness Criteria
- Future Expansion Roadmap
- Final System Requirements Summary